LangGraph v1.0.x

Section 00: Overview & Why LangGraph

Agentic AI Foundations

March 2026

LangGraph: Graph-Based Agent Orchestration

Core Idea

LangGraph is a framework for building **stateful, multi-step agent applications as directed graphs**, where:

- **Nodes** = computational steps (LLM calls, tool execution, data processing)
- **Edges** = transitions between nodes (control flow)
- **State** = shared data that flows through the graph
- **Cycles** = looping paths for agentic decision-making

Think of LangGraph as *"orchestrating conversations as graphs"*-not linear chains, but flexible, cyclic workflows.

Why Chains Aren't Enough

Limitations of Linear Pipelines

- **No cycles:** can't loop back (agents need to reason -¿ act $\rightarrow$ reason again)
- **Stateless:** each step loses context of previous decisions
- **No branching:** can't route to different paths based on conditions
- **No human-in-the-loop:** no pause points for human approval
- **Hard to debug:** linear flow obscures agent decision logic

Example: ReAct Agent

ReAct (Reasoning + Acting) is inherently cyclic: *Thought* -¿ *Action* $\rightarrow$ *Observation* $\rightarrow$ back to *Thought*. Chains can fake this with structured outputs, but can't truly express the loop.

LangGraph: Definition & Scope

LangGraph is a low-level library for orchestrating agentic workflows as directed graphs.

What You Build

- Agent loops (ReAct, ReWOO, langgraph-agent-patterns)
- Multi-turn conversations with state
- Workflows with branching, looping, fan-out/fan-in
- Persistent, resumable applications
- Human approval workflows

Philosophy

LangGraph = **low-level** orchestration primitives. Build once, use a hundred different ways. Pairs with LangChain (runnable, prompt, tool abstractions).

LangGraph vs LangChain

Aspect	LangChain	LangGraph
Level	High-level convenience	Low-level orchestration
Use case	Quick chains, RAG, prompt templates	Agents, workflows, cycles
State	Implicit (context window)	Explicit, typed, persistent
Cycles	No native support	Core feature
Control flow	Sequential (mostly)	Explicit routing & branching
Debugging	Callbacks, logging	Introspectable graphs

The Relationship

LangGraph *uses* LangChain primitives (runnables, tools, chat models). But LangGraph is the orchestration layer for **multi-step, interactive workflows**.

Key Value Propositions

5 Pillars of LangGraph

- **Cycles:** Loop back for agentic reasoning (agent patterns)
- **State:** Typed, persistent, multi-step workflows
- **Persistence:** Save/resume execution (checkpointing)
- **Human-in-the-loop:** Pause for human approval or input
- **Streaming:** Stream token-by-token output without loading full response

Why This Matters

These features unlock use cases like *multi-step planning*, *tool-use agents*, *approval workflows*, *long-running tasks*, and *complex reasoning chains*.

The Graph Mental Model: Nodes, Edges, State

Comp.	Definition
Nodes	Compute; return updates
Edges	Routes between nodes
State	Shared data
Cond. Edge	Routes on conditions
START/END	Entry/exit

Nodes-¿partial updates; graph merges. Analogy: nodes=intersections, edges=roads.

LangGraph Ecosystem

Stack

- **Core:** StateGraph, compilation, invoke
- **Checkpoint:** Persistence (save/resume)
- **Platform:** Cloud deployment, jobs
- **LangSmith:** Monitoring, evaluation

Installation & Setup

pip install

```
pip install langgraph langgraph-checkpoint
pip install langchain langchain-community
pip install python-dotenv
```

API Keys

Set environment variables for LLM providers:

```
export OPENAI_API_KEY="sk-..."
export ANTHROPIC_API_KEY="sk-ant-..."
```

Verify Installation

```
python -c "import langgraph; print(langgraph.__version__)"
# Output: 1.0.x
```

Your First LangGraph Program (Minimal)

Minimal Chatbot

```python
from langgraph.graph import StateGraph, START, END
from typing_extensions import TypedDict

class State(TypedDict):
    messages: list

def chatbot(state):
    return {"messages": state["messages"] + ["Hi"]}

graph = StateGraph(State)
graph.add_node("chatbot", chatbot)
graph.add_edge(START, "chatbot")
graph.add_edge("chatbot", END)
compiled = graph.compile()
result = compiled.invoke({"messages": []})
```

Pattern: Define -¿ Add → Compile → Invoke

Package Architecture

Package	Purpose
langgraph	Core: StateGraph, nodes, edges, compilation
langgraph.graph	Graph classes, START/END, builders
langgraph.checkpoint	Persistence backends (save/resume)
langgraph-platform	Cloud deployment, workers
langgraph-supervisor	Multi-agent orchestration

Common Imports

```
from langgraph.graph import StateGraph, START, END
```

Core Concepts Preview Table

Concept	Type	Description
`State`	TypedDict	Shared data; merged via reducers
`Reducer`	Function	Merges state updates
`Node`	Function	Computes state updates
`Edge`	Connection	Routes node-to-node
`Conditional Edge`	Router	Routes based on state
`START / END`	Special	Entry/exit points
`StateGraph`	Class	Orchestrates graph
`Compiled Graph`	Runnable	Has `invoke()`, `stream()`

Each is fundamental. Sections 01-03 dive deep.

LangGraph vs Alternatives

	LangGraph	**AutoGen**	**CrewAI**	**Raw**
State	Typed	Implicit	Implicit	Ad-hoc
Cycles	Native	Yes	Yes	Manual
Persist	Yes	No	No	No
Debug	Great	Good	Good	Hard
Flexibility	Very high	Med	Med	Highest

When to Use

- Fine control over flow, persistence, debugging

Learning Path Ahead

Sections 00-03

- **Sec 00:** Overview
- **Sec 01:** State, nodes, edges
- **Sec 02:** Building graphs
- **Sec 03:** Conditional routing

Beyond: persistence, tools, streaming, deployment.

Summary & Cheat Sheet

Key Points

- Graph-based orchestration (cycles, state, persistence)
- Chains can't loop; graphs can
- Core: nodes, edges, state
- Pattern: define -¿ add → compile → invoke

Sequential = LangChain. Looping/agentic = LangGraph.

Self-Check Questions

1. Why can't linear chains express agent loops?
2. What are the three core components of LangGraph?
3. What is a reducer and why does state need one?
4. How does LangGraph differ from LangChain?
5. Name two value propositions.
6. What is the basic workflow to create and run?
7. When would you choose LangGraph over raw code?

Quick Answers: (1) Chains are linear. (2) Nodes, edges, state. (3) Merges updates; tracks state. (4) Low-level vs high-level. (5) Cycles, persistence. (6) Define -¿ add $\rightarrow$ compile $\rightarrow$ invoke. (7) Complex workflows.